Universidad

Ingeniería de Sistemas y Computación

Asignatura: Redes Neuronales Artificiales

Parcial Momento Uno

Laboratorio TensorFlow Playground

Estudiante: Óscar Mauricio García Mesa

Fecha: 13 de marzo de 2026

Herramienta utilizada: `https://playground.tensorflow.org/`

Índice

1. Introducción **2**

2. Marco Teórico **2**
2.1. Componentes de una red neuronal 2
2.2. Hiperparámetros clave 2

3. Ejercicios de Clasificación **3**
3.1. Ejercicio 1 — Clasificación lineal básica 3
3.2. Ejercicio 2 — Clasificación no lineal 7
3.3. Tabla resumen de ejercicios 10
3.4. Conclusiones generales 10

Referencias **10**

1. Introducción

Las redes neuronales artificiales son modelos de aprendizaje automático inspirados en el funcionamiento del cerebro humano. Estos modelos aprenden patrones en los datos mediante un proceso de entrenamiento iterativo que ajusta automáticamente sus parámetros internos (pesos y sesgos) a través del algoritmo de retropropagación del gradiente.

En este laboratorio se utilizó la herramienta interactiva **TensorFlow Playground** (`https://playground.tensorflow.org/`) para experimentar visualmente con distintas configuraciones de redes neuronales y observar en tiempo real cómo influyen parámetros como el número de capas, la función de activación y la tasa de aprendizaje en la capacidad del modelo para aprender patrones en datos de clasificación y regresión.

El laboratorio consta de **10 ejercicios**: 5 de clasificación y 5 de regresión, organizados de menor a mayor complejidad.

2. Marco Teórico

2.1. Componentes de una red neuronal

Una red neuronal artificial está compuesta por:

- **Capa de entrada:** recibe las variables de entrada (features). No realiza transformaciones, solo distribuye los datos a la siguiente capa.
- **Capas ocultas:** realizan transformaciones no lineales sobre los datos. Cada neurona calcula una suma ponderada de sus entradas y aplica una función de activación: $a = f(\sum_i w_i x_i + b)$.
- **Capa de salida:** produce la predicción final. En clasificación binaria usa sigmoide; en multiclase usa softmax; en regresión usa función lineal.

2.2. Hiperparámetros clave

- **Learning rate (α):** controla el tamaño del paso en la actualización de pesos: $w \leftarrow w - \alpha \nabla L$. Un valor muy alto causa divergencia; muy bajo, convergencia lenta.
- **Épocas:** número de veces que la red procesa el conjunto completo de entrenamiento.
- **Overfitting:** cuando el modelo memoriza los datos de entrenamiento en lugar de generalizar. Se detecta cuando *training loss* $\ll$ *test loss*.
- **Underfitting:** modelo demasiado simple para capturar el patrón. Ambos losses permanecen altos.

3. Ejercicios de Clasificación

3.1. Ejercicio 1 — Clasificación lineal básica

Problema 1 del parcial: Identificación de clientes en dos segmentos.

Descripción

Una empresa de comercio electrónico desea clasificar automáticamente sus usuarios en dos segmentos según su comportamiento digital. El dataset *Two clusters* representa dos grupos de puntos claramente separados en el espacio bidimensional.

Configuración en TensorFlow Playground

- **Dataset:** Two clusters
- **Hidden layers:** 1 capa oculta
- **Neuronas:** 1
- **Activation:** Tanh → ReLU → Sigmoid (comparar)
- **Learning rate:** 0.03 → 0.0001 → 0.3 (comparar)
- **Problem type:** Classification

Proceso y evidencias

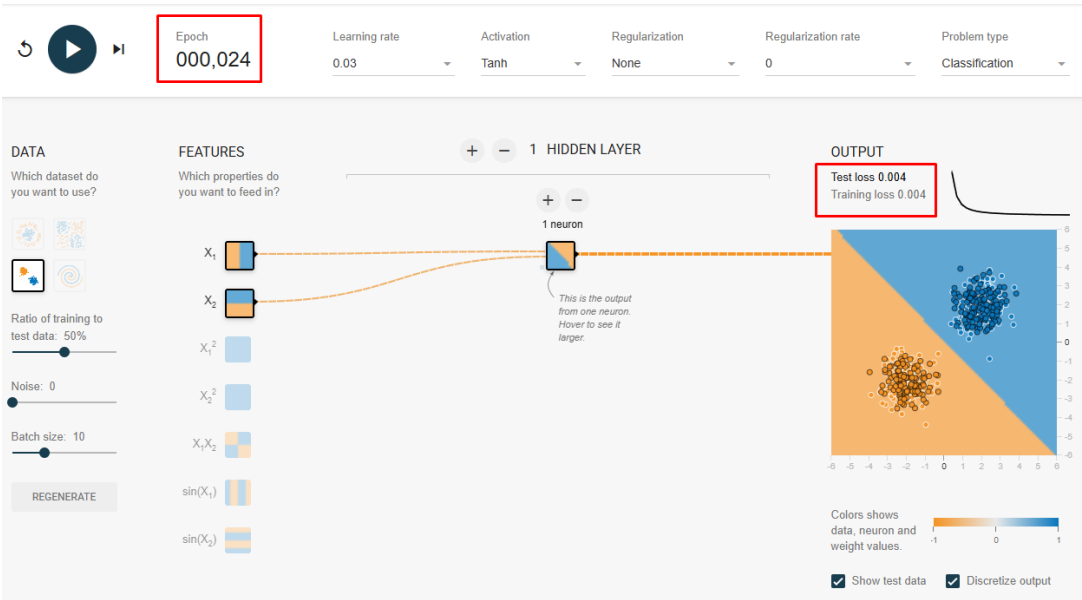

Figura 1: Tanh, 1 nuerona, rate: 0.03. Epochs 24, Test loss = 0.004

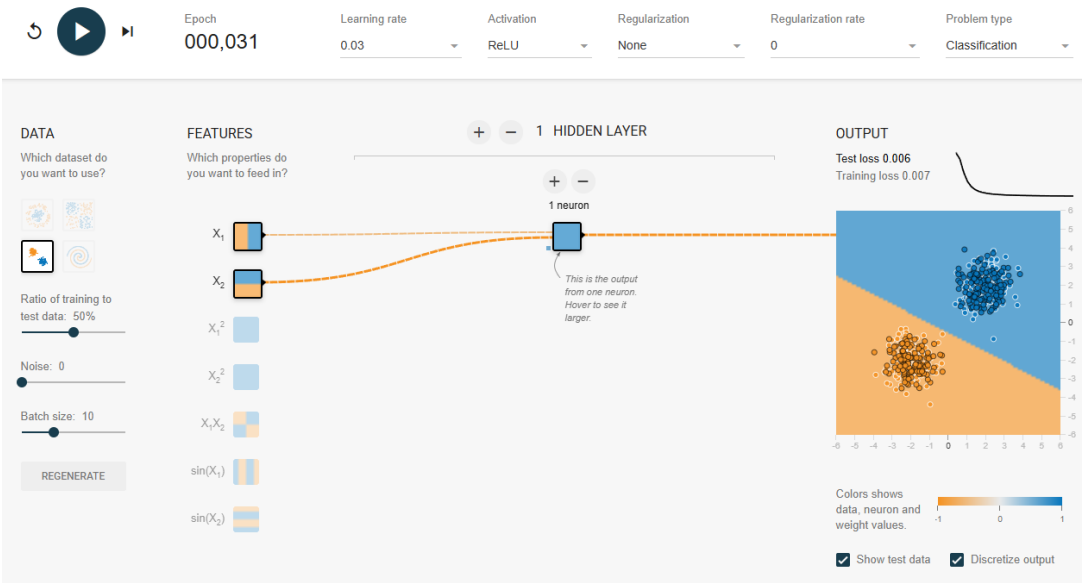

Figura 2: ReLu, 1 nuerona, rate: 0.03, Epocas 31 = test loss = 0.006

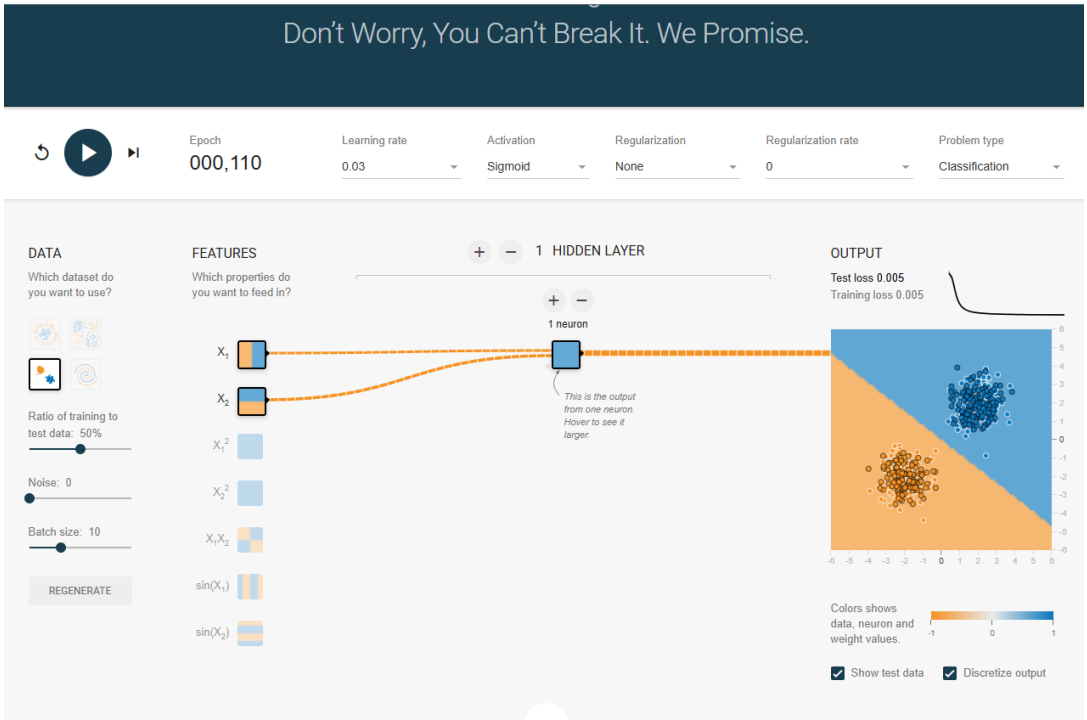

Figura 3: Sigmoid, 1 nuerona, rate: 0.03, epochs

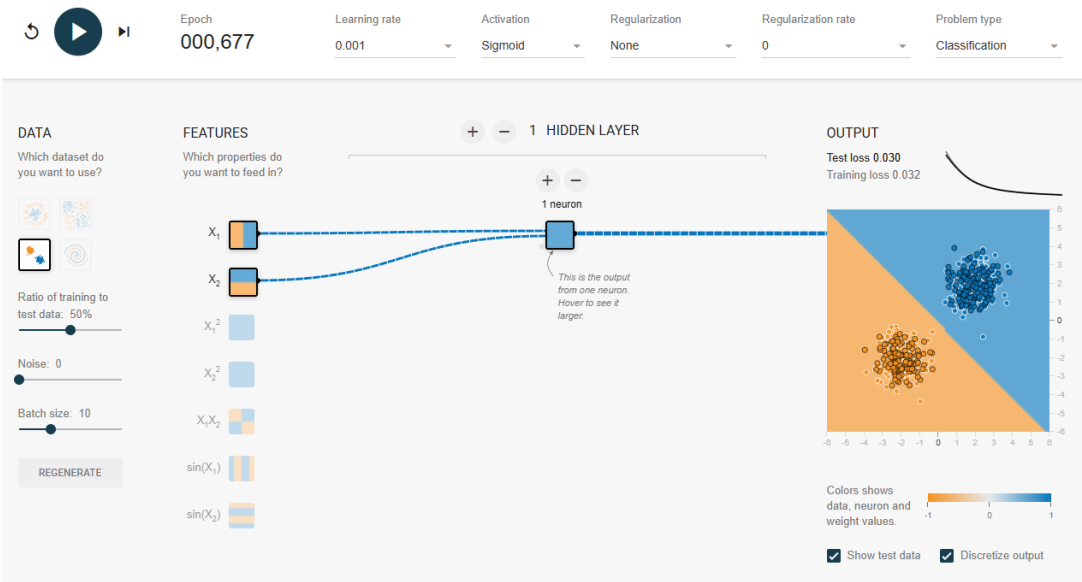

Figura 4: Sigmoid, 1 nuerona, rate: 0.001

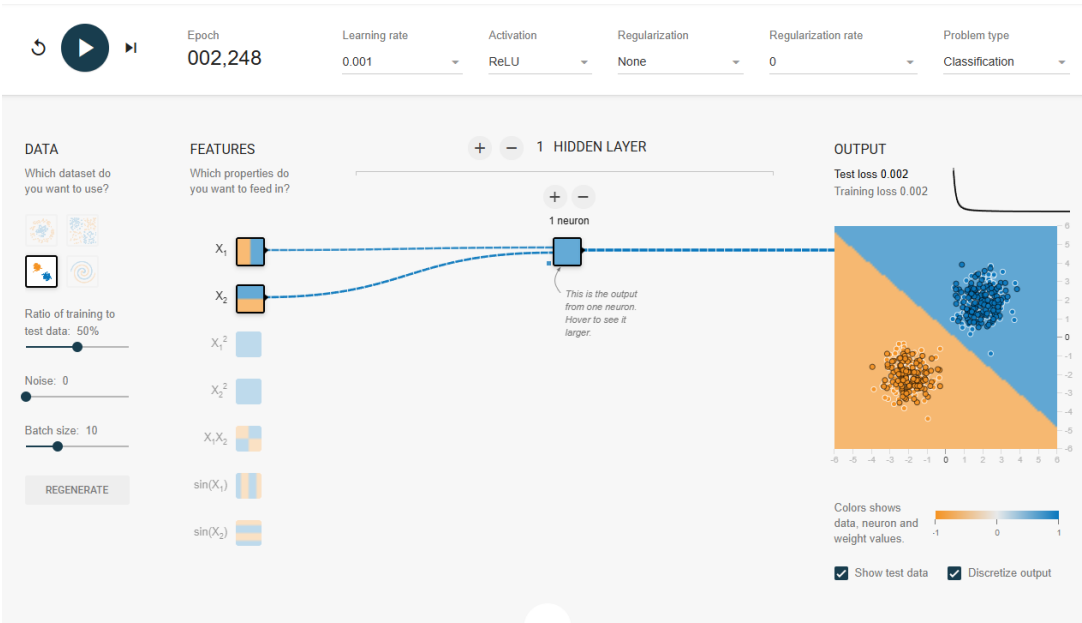

Figura 5: Sigmoid, 1 nuerona, rate: 0.03, epochs

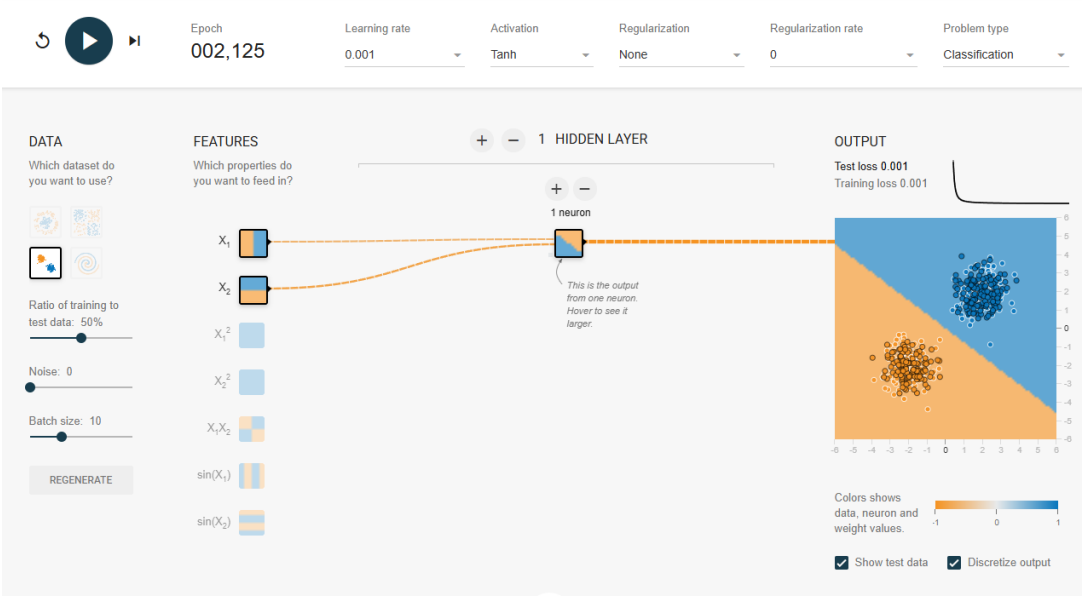

Figura 6: Sigmoid, 1 nuerona, rate: 0.001

Respuestas y análisis

¿Con qué función de activación aprende más rápido?
Las funciones **Tanh** y **ReLU** convergen más rápido que Sigmoid, típicamente antes de las 200 épocas. Tanh tiene rango $(-1, 1)$ y es simétrica alrededor de cero, lo que facilita el flujo del gradiente. ReLU es computacionalmente simple ($\text{máx}(0, x)$) y evita la saturación. Sigmoid se satura en sus extremos (salidas cercanas a 0 o 1), haciendo que el gradiente tienda a cero y ralentizando el aprendizaje.

¿Qué pasa cuando el learning rate es muy alto (0.3)?
El modelo **oscila o diverge**: el loss sube y baja sin estabilizarse porque los pasos de actualización de pesos son tan grandes que el algoritmo "salta" el mínimo de la función de pérdida repetidamente, en lugar de descender hacia él gradualmente.

¿Cuántas épocas tarda en converger?
Con Tanh y lr=0.03, el dataset Two clusters converge aproximadamente en **100–300 épocas**. Con lr=0.0001 puede requerir más de 5.000 épocas para alcanzar el mismo nivel de precisión.

3.2. Ejercicio 2 — Clasificación no lineal

Problema 2 del parcial: Detección de una zona geográfica circular de cobertura.

Descripción

Una empresa de logística necesita determinar si un punto geográfico está dentro o fuera de una zona de cobertura circular. El dataset *Circle* presenta puntos azules en el centro y puntos naranjas en el anillo exterior — una frontera circular que ninguna línea recta puede separar.

Configuración en TensorFlow Playground

- **Dataset:** Circle
- **Hidden layers:** 1 capa oculta
- **Neuronas:** 3 (luego comparar con 1 y con 6)
- **Activation:** Tanh
- **Learning rate:** 0.03

Proceso y evidencias

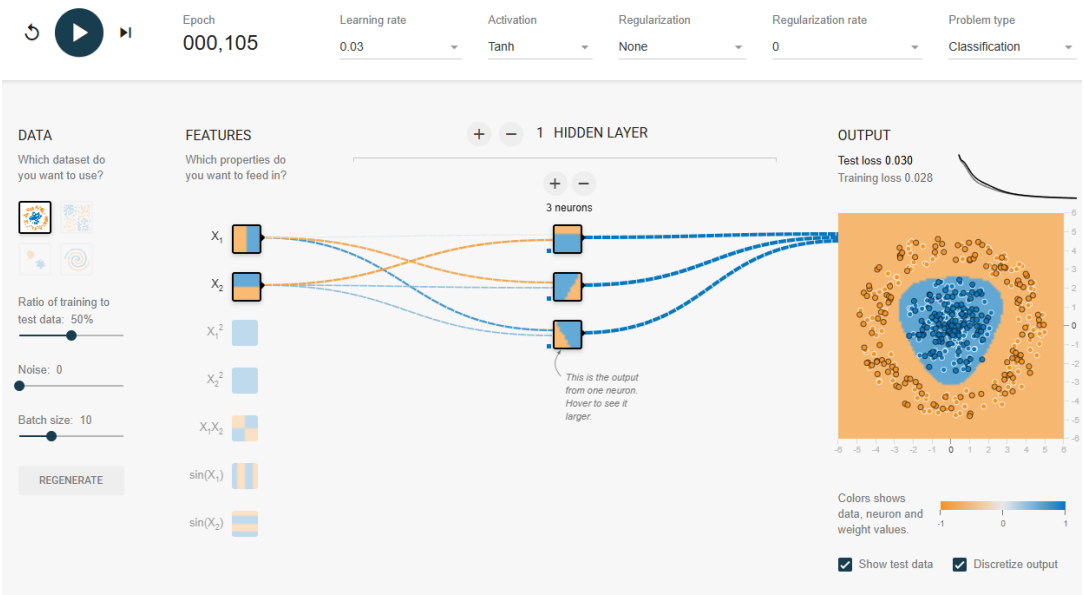

Figura 7: Configuración inicial, 3 neuronas

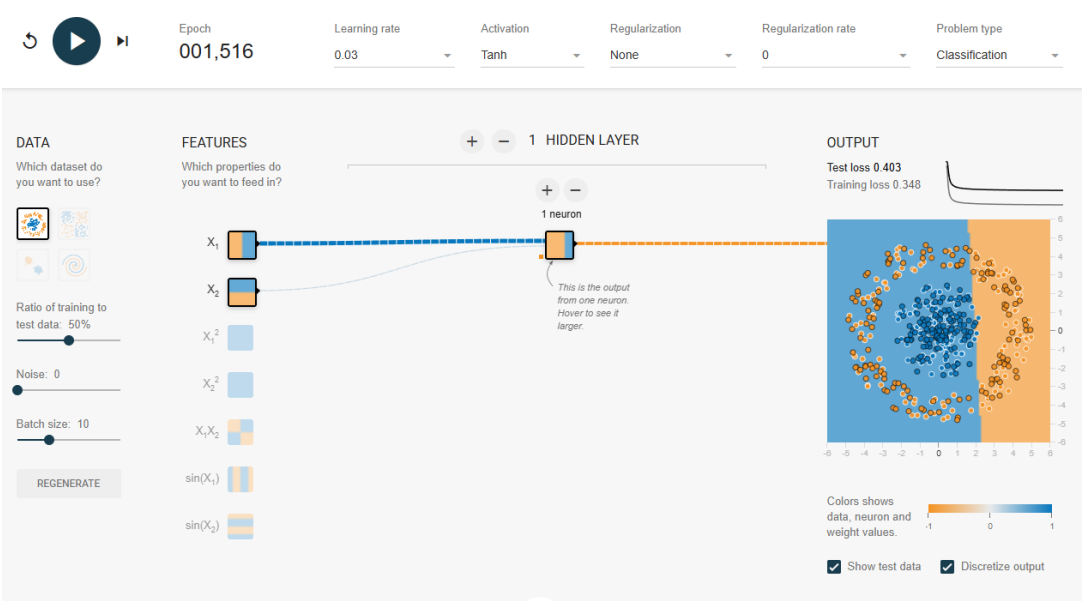

Figura 8: Eliminación de dos neuronas. Con solo 1 neurona la red solo puede aprender una frontera lineal, incapaz de separar los puntos del círculo. Loss alto que no converge al valor esperado.

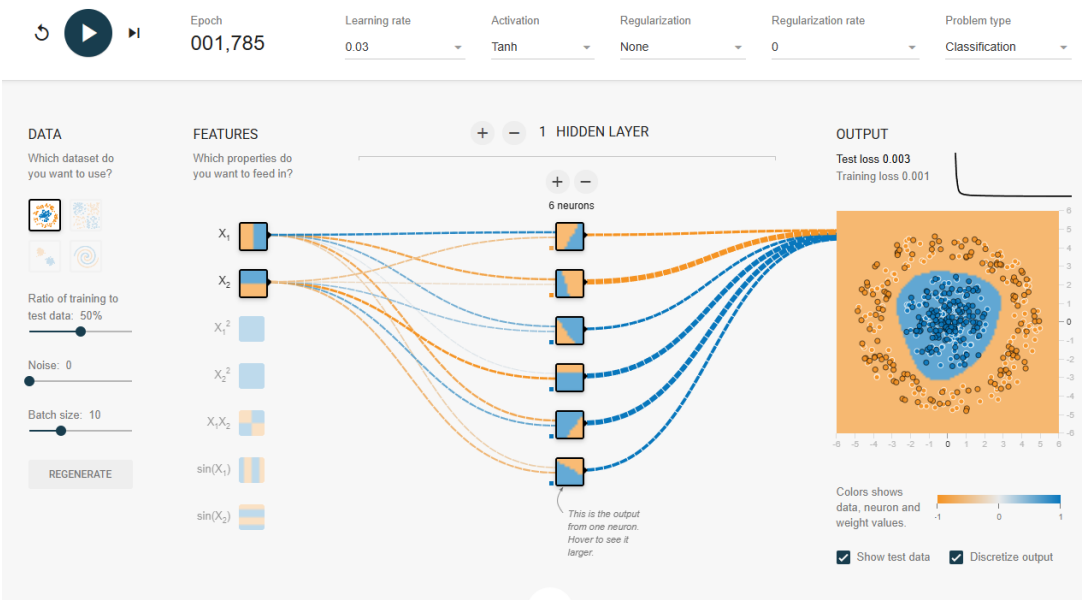

Figura 9: Con 6 neuronas: la frontera circular es notablemente más precisa. El loss es significativamente menor que con 1 o 3 neuronas.

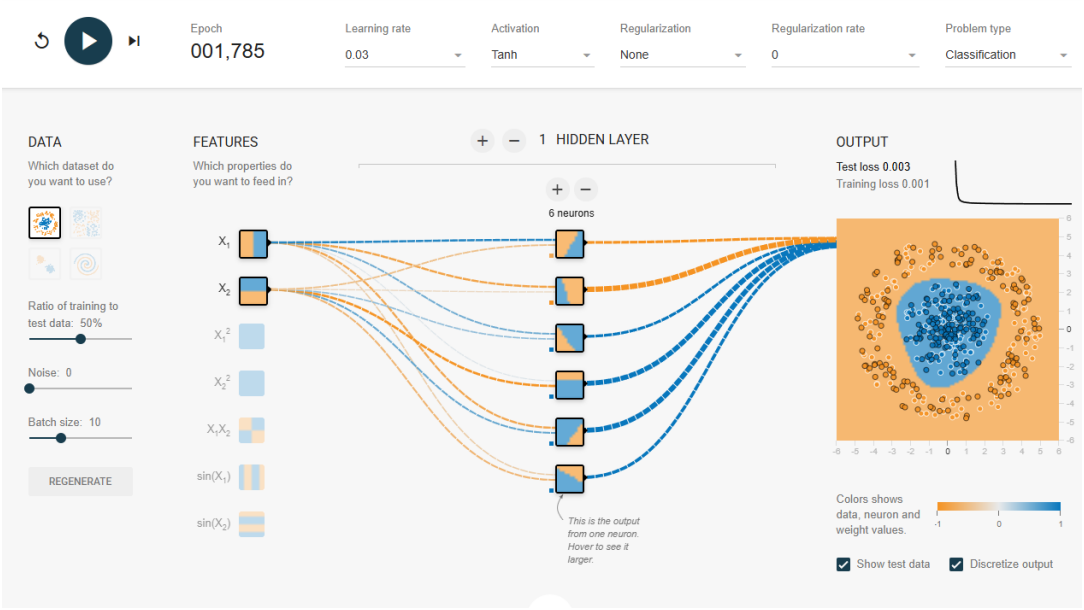

Figura 10: Con 6 neuronas y 2 capas ocultas: frontera aún más suave y precisa. Comparar el loss con la configuración de 1 capa.

Respuestas y análisis

¿Qué ocurre con pocas neuronas (1 neurona)?
Con una sola neurona la red **solo puede aprender una frontera lineal** — una línea recta que divide el plano. El loss permanece alto y la clasificación es incorrecta para muchos puntos.

¿Cómo cambia la frontera de decisión?
Con 3 neuronas aparece una curva que aproxima el círculo, aunque de forma irregular. Con 6 neuronas la frontera se vuelve notablemente más circular. Cada neurona aprende una "porción" de la curva; combinadas, reconstruyen la forma circular.

¿Qué ocurre si se agrega una segunda capa oculta?
La segunda capa combina las representaciones aprendidas por la primera, permitiendo capturar relaciones aún más complejas. La frontera se vuelve más suave y precisa, y el loss converge a valores menores. Esto ilustra el principio de las redes *profundas*: la jerarquía de capas permite construir representaciones cada vez más abstractas.

¿Qué significa añadir nuevas características?
Agregar features es **expandir el espacio de representación de los datos**, proporcionando a la red perspectivas adicionales sobre los mismos datos originales. x_1^2 informa sobre la variación cuadrática; $x_1 \cdot x_2$ informa sobre la interacción entre variables. Sin embargo, agregar features irrelevantes puede introducir ruido y empeorar el modelo — la selección inteligente de features (*feature engineering*) es una de las habilidades más valiosas en ML.

3.3. Tabla resumen de ejercicios

3.4. Conclusiones generales

A partir de los 10 ejercicios realizados en TensorFlow Playground, se pueden extraer las siguientes conclusiones:

1. **La complejidad del modelo debe ser proporcional a la complejidad del problema.** Una red con demasiadas neuronas o capas para un problema simple conducirá a overfitting; una red muy simple para un problema no lineal conducirá a underfitting.
2. **El learning rate es uno de los hiperparámetros más críticos.** Un valor adecuado (típicamente entre 0.001 y 0.03) garantiza convergencia estable. Valores extremos causan entrenamiento lento o divergencia.
3. **Las features de entrada son determinantes.** Ninguna arquitectura de red puede compensar la falta de información relevante en la entrada. El feature engineering puede ser más impactante que agregar más capas.
4. **Los datos son el recurso más valioso.** Como demostró el ejercicio de Spiral, la cantidad y calidad de los datos de entrenamiento tiene un impacto directo en la capacidad de generalización del modelo.
5. **El test loss es la métrica honesta.** Siempre evaluar el modelo con datos que no participaron en el entrenamiento. Un training loss bajo con test loss alto es señal inequívoca de overfitting.

Referencias

1. Smilkov, D. & Carter, S. (2016). *TensorFlow Playground*. Google Brain. `https://playground.tensorflow.org/`
2. Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press. `http://www.deeplearningbook.org/`
3. Nielsen, M. (2015). *Neural Networks and Deep Learning*. Determination Press. `http://neuralnetworksanddeeplearning.com/`
4. Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.